

Yudillo 백엔드 — 실행 흐름과 코드 리뷰

작성일: 2026-09-28 대상: `C:\Users\PC\Desktop\참고자료\Study\back-end-develop` 범위: 인증(auth) + 사용자(user). 칸반 도메인은 프론트 화면이 생길 때 추가 예정

이전에 작성한 백엔드 가이드 문서는 프로젝트/리스트/카드 도메인이 있던 시점의 설명이라
지금 코드와 다르다. 이 문서가 **현재 코드 기준**이다.

0. 이 문서의 구성

장	내용
1	현재 프로젝트 스냅샷 — 파일 구성, 규모
2	실행 흐름 — 기동부터 각 API 의 단계별 동작
3	코드 리뷰 — 잘 된 점, 고칠 점, 죽은 코드
4	권장 작업 순서

2장은 "무엇이 어떤 순서로 일어나는가", 3장은 "그게 잘 짜였는가"를 다룬다. 코드를 옆에 열어두고 읽는 것을 전제로 썼다.

1. 현재 프로젝트 스냅샷

1.1 규모

자바 파일 44개 (테스트 1개 포함)
총 라인 1,939줄
API 7개
DB 테이블 2개 (+ flyway 이력 테이블)

1.2 파일 구성

src/main/java/com/yudillo/api/	
├─ ApiApplication.java	진입점
├─ global/	공통 관심사 (26개)
│ ├─ config/	
│ │ ├─ SecurityConfig	필터 체인, 경로 권한, CORS, BCrypt
│ │ ├─ SwaggerConfig	문서 메타정보, bearerAuth 스킴
│ │ ├─ JacksonConfig	Instant 직렬화 형식
│ │ ├─ DateTimeProperties	app.datetime 설정 바인딩
│ │ ├─ CorsProperties	app.cors 설정 바인딩
│ │ ├─ ApiErrorCodes	(애노테이션) 에러 문서화
│ │ ├─ ErrorResponseCustomizer	에러 응답 문서 자동 생성
│ │ ├─ ApiSuccessMessage	(애노테이션) 성공 문구 문서화
│ │ └─ SuccessResponseCustomizer	성공 응답 예시 자동 생성
│ └─ security/	
│ ├─ JwtTokenProvider	토큰 발급/검증 (서명 키를 다루는 유일한 곳)
│ ├─ JwtProperties	jwt.* 설정 바인딩
│ ├─ JwtAuthenticationFilter	요청마다 토큰 검사
│ ├─ AuthUser	인증 주체 (UserDetails 구현)
│ ├─ AuthUserDetailsService	DB 에서 사용자 조회
│ ├─ CurrentUser	(애노테이션) 현재 사용자 주입
│ ├─ TokenType	ACCESS / REFRESH
│ ├─ JwtAuthenticationException	토큰 예외 (한 종류로 통합)
│ ├─ JwtAuthenticationEntryPoint	401 JSON 응답
│ └─ JwtAccessDeniedHandler	403 JSON 응답
│ └─ exception/	
│ ├─ ErrorCode	모든 에러 코드 (12개)
│ ├─ BusinessException	서비스가 던지는 예외
│ ├─ ErrorResponse	에러 응답 포맷
│ └─ GlobalExceptionHandler	예외 → HTTP 응답
│ └─ response/	
│ ├─ ApiResponse	성공 응답 껍데기
│ ├─ SuccessMessage	성공 문구 (5개)
│ └─ PageResponse	페이지네이션 (현재 미사용)
│ └─ entity/	
│ └─ BaseTimeEntity	createdAt / updatedAt 자동화
├─ auth/	인증 (8개)
│ ├─ controller/AuthController	signup / login / reissue / logout
│ ├─ service/AuthService	인증 업무 규칙
│ ├─ entity/RefreshToken	Refresh 토큰 저장
│ ├─ repository/RefreshTokenRepository	
│ └─ dto/ SignupRequest, LoginRequest, ReissueRequest, TokenResponse	
└─ user/	사용자 (8개)
│ ├─ controller/UserController	me / me/name / me/password
│ ├─ service/UserService	
│ ├─ entity/User, Role	
│ ├─ repository/UserRepository	
│ └─ dto/ UserResponse, UpdateNameRequest, ChangePasswordRequest	

1.3 API 목록

Method	Path	인증	성공
POST	/api/auth/signup	-	201
POST	/api/auth/login	-	200
POST	/api/auth/reissue	-	200
POST	/api/auth/logout	필요	200
GET	/api/users/me	필요	200
PATCH	/api/users/me/name	필요	200
PATCH	/api/users/me/password	필요	200

1.4 DB

```
users
  id uuid PK, email unique, password(BCrypt), name, role,
  email_verified, created_at, updated_at

refresh_tokens
  id bigserial PK, user_id FK→users(cascade), token unique,
  expires_at, created_at
```

`name` 은 중복을 허용한다. 식별자는 `email` 이다.

2. 실행 흐름

2.1 애플리케이션 기동 순서

`./gradlew bootRun` 또는 F5 를 누르면 이 순서로 진행된다. 순서가 중요하다.

① 프로필 결정

`spring.profiles.default: local` → `application.yml` + `application-local.yml`

② `@ConfigurationProperties` 바인딩 + 검증

`JwtProperties` `jwt.secret` 이 비면 여기서 기동 실패 (`@NotBlank`)
`DateTimeProperties` 잘못된 패턴이면 `formatter()` 에서 실패
`CorsProperties`

③ `DataSource` 생성 (`HikariCP`)

DB 가 안 떠 있으면 여기서 실패

④ Flyway 실행

← JPA 보다 먼저

`flyway_schema_history` 를 읽어 미적용 버전만 실행
V1 → V2 순서

⑤ `Hibernate ddl-auto: validate`

← Flyway 가 만든 스키마와 엔티티 대조

불일치하면 기동 실패

⑥ 빈 생성

`SecurityConfig` → `SecurityFilterChain` 구성
`JacksonConfig` → `Instant` 직렬화기를 `JsonMapper` 에 등록
...

⑦ 내장 Tomcat 기동 (`SERVER_PORT`, 기본 8080)

⑧ 첫 `/v3/api-docs` 요청 시 OpenAPI 문서 생성 (이후 캐시)

`OperationCustomizer` → 컨트롤러 애노테이션 읽어 응답 정의 추가
`OpenApiCustomizer` → 스키마 등록, 성공 예시 생성

④와 ⑤의 순서가 핵심이다. Flyway 가 테이블을 만든 다음 JPA 가 검증한다. 반대였다면 첫 기동에서 무조건 실패한다.

2.2 회원가입 — `POST /api/auth/signup`

인증이 필요 없는 경로다.

요청

```
{ "email": "user@example.com", "password": "user123", "name": "홍길동" }
```



① JwtAuthenticationFilter

Authorization 헤더 없음 → 아무것도 안 하고 통과
(여기서 401 을 던지지 않는 게 중요. 던지면 로그인 자체가 불가능해진다)



② AuthorizationFilter

PUBLIC_ENDPOINTS 에 /api/auth/signup 이 있음 → permitAll 통과



③ @Valid 검증 (SignupRequest)

email @NotBlank @Email @Size(max=255)
password @NotBlank @Pattern(영문+숫자+특수문자 8~20자)
name @NotBlank @Pattern(한글/영문/숫자 2~10자)
실패 → MethodArgumentNotValidException → 400 C001 (+ errors 배열)



④ AuthController.signup()

@ResponseStatus(CREATED) 로 201 예약



⑤ AuthService.signup()

@Transactional 시작

| existsByEmail() → true 면 409 U002, 롤백
| passwordEncoder.encode() BCrypt 해싱 (salt 자동 포함)
| User.builder() 메모리에만 존재
| userRepository.save() INSERT
이 시점에 id(UUID), created_at, updated_at 이 채워진다



⑥ UserResponse.from(user)

password 를 제외한 DTO 로 변환



▼ 커밋

⑦ ApiResponse.ok(data, SuccessMessage.SIGNUP)



201 Created

```
{
  "success": true,
  "data": { "id": "...", "email": "...", "name": "홍길동",
            "role": "USER", "emailVerified": true, "createdAt": "2026-09-28 11:45:11" },
  "message": "회원가입이 완료되었습니다."
}
```

실행되는 SQL 은 정확히 2개다.

```
select count(*) from users where email = ?      -- ⑤ existsByEmail
insert into users (...) values (...)             -- ⑤ save
```

2.3 로그인 — POST /api/auth/login

```
요청 { "email": "...", "password": "..." }
|
▼
AuthService.login() @Transactional
├ findByEmail()
│   없으면 → 401 A002 INVALID_CREDENTIALS
├ passwordEncoder.matches(평문, 저장된 해시)
│   불일치 → 401 A002 INVALID_CREDENTIALS ← 위와 같은 에러
└ issueTokens(user)
    ├── createAccessToken(user) 30분, type=ACCESS, role/email 포함
    ├── createRefreshToken(user) 14일, type=REFRESH, jti 포함
    ├── deleteByUserId() 기존 토큰 폐기 (사용자당 1개 정책)
    └ save(RefreshToken) 새 토큰을 DB 에 기록
|
▼
200 { accessToken, refreshToken, tokenType: "Bearer", expiresIn: 1800 }
```

두 실패 경우에 같은 에러를 주는 것이 의도다. 구분하면 공격자가 "이 이메일은 가입되어 있음"을 알아낼 수 있다(사용자 열거). 그 목록은 피싱이나 무차별 대입의 출발점이 된다.

Access / Refresh 를 나눈 구조

	Access	Refresh
수명	30분	14일
DB 저장	X	O
검증	서명만 (무상태)	서명 + DB 존재 확인
폐기	불가 (만료 대기)	즉시 가능
용도	모든 API 호출	재발급 요청만

JWT 는 발급하면 취소할 수 없다는 게 근본 한계다. Access 를 짧게 해 노출 시간을 줄이고, 긴 수명이 필요한 Refresh 만 DB 에 두어 폐기 가능하게 만든 균형이다.

2.4 인증이 필요한 요청 — 필터 동작

GET /api/users/me 를 예로 든다.

요청

GET /api/users/me

Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...

① CorsFilter

Origin 이 허용 목록에 있는지

② JwtAuthenticationFilter (OncePerRequestFilter)

```
├ resolveToken()
│   "Bearer " 접두어 확인 후 뒷부분 추출
│   없으면 → 그냥 통과 (뒤에서 막힘)
├ tokenProvider.parse(token, ACCESS)
│   ├── 서명 검증      위조 차단
│   ├── 만료 검증      지났으면 ExpiredJwtException
│   ├── 발급자 검증    requireIssuer
│   └── 타입 검증      REFRESH 를 여기 쓰면 거부
├ userDetailsService.loadById(uuid)
│   DB 에서 사용자 조회 → AuthUser 생성
├ UsernamePasswordAuthenticationToken.authenticated(principal, null, authorities)
│   credentials 를 null 로 둔다 (비밀번호를 메모리에 안 남김)
└ SecurityContextHolder 에 저장      ← ThreadLocal
    실패 시 clearContext() 후 EntryPoint 로 401
```

③ AuthorizationFilter

anyRequest().authenticated() → SecurityContext 가 비었으면 401

④ DispatcherServlet → UserController.getMe()

@CurrentUser 가 SecurityContext 에서 AuthUser 를 꺼내 주입

⑤ userService.getMe(authUser.getId()) ← AuthUser 가 아니라 UUID 를 넘긴다

200 { "success": true, "data": { ... } }

이 흐름에서 눈여겨볼 3가지

토큰이 없어도 필터는 통과시킨다. 필터의 역할은 "누구인지 확정"까지이고, "이 경로에 인증이 필요한가"는 뒤의 `AuthorizationFilter` 가 판단한다. 필터에서 바로 401 을 던지면 `/api/auth/login` 같은 공개 경로가 동작하지 못한다.

매 요청마다 DB 를 조회한다. 토큰 payload 에 `role`, `email` 이 있으니 DB 없이 `AuthUser` 를 만들 수도 있다. 그럼에도 조회하는 이유는 탈퇴·차단·권한 변경이 **즉시 반영**되어야 하기 때문이다. 토큰만 믿으면 최대 30 분 지연된다.

서비스에는 UUID 만 넘긴다. `AuthUser` 는 Spring Security 의 `UserDetails` 구현체다. 서비스가 이걸 받으면 시큐리티에 의존하게 되고, 테스트마다 `AuthUser` 를 만들어야 한다. 값 하나만 넘기면 배치 작업에서도 같은 서비스를 부를 수 있다.

2.5 토큰 재발급 — POST /api/auth/reissue

검증이 3단이다.

```
요청 { "refreshToken": "..." }
|
▼
AuthService.reissue()
  ① tokenProvider.parse(token, REFRESH)
    서명 · 만료 · 발급자 · 타입 검증
  ② refreshTokenRepository.findByToken()
    DB 에 없으면 → 401 A003
    ★ 이 단계가 "폐기 기능"의 실체다.
    로그아웃했거나 이미 교체된 토큰은 여기서 걸린다.
  ③ saved.isExpired() || userId 불일치
    이상하면 그 행을 삭제하고 401
  ④ 새 Access + 새 Refresh 발급
  ⑤ saved.rotate(newRefreshToken, newExpiresAt)
    같은 행의 값을 교체 → 커밋 시 UPDATE
    ★ 옛 Refresh 토큰은 이 순간 무효가 된다 (rotation)
|
▼
200 { 새 accessToken, 새 refreshToken, ... }
```

Rotation 이 막는 것

공격자가 Refresh 토큰을 탈취했다고 가정하면,

- 공격자가 먼저 쓰면 → 정상 사용자의 재발급이 실패해 **이상을 감지**할 수 있다
- 정상 사용자가 먼저 쓰면 → 공격자의 토큰이 **즉시 무효**가 된다

어느 쪽이든 피해 시간이 짧아진다. Rotation 없이 재사용을 허용하면 탈취된 토큰이 14일간 살아있다.

2.6 로그아웃 — POST /api/auth/logout

```
AuthService.logout(userId)
└ refreshTokenRepository.deleteByUserId(userId)
```

한 줄이다. **Access 토큰은 만료까지 유효하다.** 서버가 상태를 안 들고 있어서 이미 발급한 Access 토큰을 취소할 방법이 없다. 최대 30분간 그 토큰으로 API 를 쓸 수 있다.

실무에서는 세 가지로 대응한다.

1. Access 수명 단축 (현재 30분 → 5분)
2. `jti` 블랙리스트를 Redis 에 두고 필터에서 확인 (무상태 일부 포기)
3. 클라이언트가 저장된 토큰을 삭제 (대부분의 실제 로그아웃은 이걸로 충분)

2.7 비밀번호 변경 — PATCH /api/users/me/password

```
UserService.changePassword() @Transactional
├─ findUser(userId)
├─ matches(현재비밀번호, 저장된해시)
│   └─ 불일치 → 400 U004 PASSWORD_MISMATCH
├─ matches(새비밀번호, 저장된해시)
│   └─ 같으면 → 400 U005 SAME_PASSWORD
├─ user.changePassword(encode(새비밀번호))
│   └─ 변경 감지로 UPDATE (save() 호출 없음)
└─ refreshTokenRepository.deleteByUserId()
    ★ 모든 Refresh 토큰 폐기
```

마지막 줄이 중요하다. 비밀번호를 바꾸는 이유는 대개 **유출 의심**이다. 기존 Refresh 토큰이 살아있으면 비밀번호를 훔친 사람이 14일간 계속 접근한다. 토큰을 지우면 그 사람도 재로그인해야 하고, 새 비밀번호를 모르니 끊긴다.

부작용으로 정상 사용자도 다른 기기에서 로그아웃된다. 그래서 응답 메시지로 안내한다.

"비밀번호가 변경되었습니다. 다시 로그인해 주세요."

save() 가 없는데 UPDATE 가 나가는 이유

JPA 의 **변경 감지(dirty checking)** 다. 트랜잭션 안에서 조회한 엔티티는 영속성 컨텍스트가 관리하고, 커밋 시점에 "조회 당시 스냅샷"과 비교해 달라진 필드만 UPDATE 한다. `updateName()` 도 같은 원리다.

2.8 에러 처리 흐름

에러가 어디서 발생했느냐에 따라 **처리 주체가 다르다**. 이게 이 프로젝트에서 가장 헷갈리기 쉬운 지점이다.

```
[A] 필터 단계에서 실패 (토큰 없음/위조/만료)
    JwtAuthenticationFilter
        → catch (AuthenticationException)
        → JwtAuthenticationEntryPoint.commence()
        → 401 JSON 직접 작성
    ★ 컨트롤러에 도달하지 못하므로 @RestControllerAdvice 가 못 잡는다

[B] 인가 실패 (권한 부족)
    AuthorizationFilter
        → JwtAccessDeniedHandler.handle()
        → 403 JSON 직접 작성
    현재는 /actuator/** 에 ADMIN 이 아닌 사용자가 접근할 때만 발생

[C] 컨트롤러/서비스에서 실패
    BusinessException 등
        → GlobalExceptionHandler (@RestControllerAdvice)
        → ErrorCode 에서 상태·코드를 꺼내 ErrorResponse 생성
```

A/B 와 C 의 응답 포맷을 일부러 똑같이 맞춰냈다. 프론트 입장에서는 어디서 실패했든 같은 모양의 JSON 이 온다.

```
{
  "success": false,
  "code": "A002",
  "message": "이메일 또는 비밀번호가 올바르지 않습니다.",
  "timestamp": "2026-09-28 13:08:16"
}
```

`GlobalExceptionHandler` 의 핸들러 5개:

핸들러	잡는 상황	응답	로그
<code>BusinessException</code>	의도적으로 던진 업무 예외	ErrorCode 대로	<code>debug</code>
<code>MethodArgumentNotValidException</code>	<code>@Valid</code> 실패	400 + <code>errors</code> 배열	없음
<code>AccessDeniedException</code>	메서드 보안 거부	403	없음
<code>HttpRequestMethodNotSupportedException</code>	POST 경로에 GET	405	없음
<code>Exception</code>	예상 못 한 전부	500 (내용 숨김)	<code>error</code> + 스택

로그 레벨을 나눈 이유가 있다. `BusinessException` 은 **정상적인 흐름**이다. 중복 이메일로 가입을 시도한 건 버그가 아니다. `error` 로 찍으면 로그가 노이즈로 가득 차고 진짜 장애가 묻힌다.

맨 아래 `Exception` 핸들러는 **보안 장치**다. 없으면 예상 못 한 예외에서 스택트레이스가 노출될 수 있다. 거기엔 내부 패키지 구조, 라이브러리 버전, SQL 일부가 들어있다.

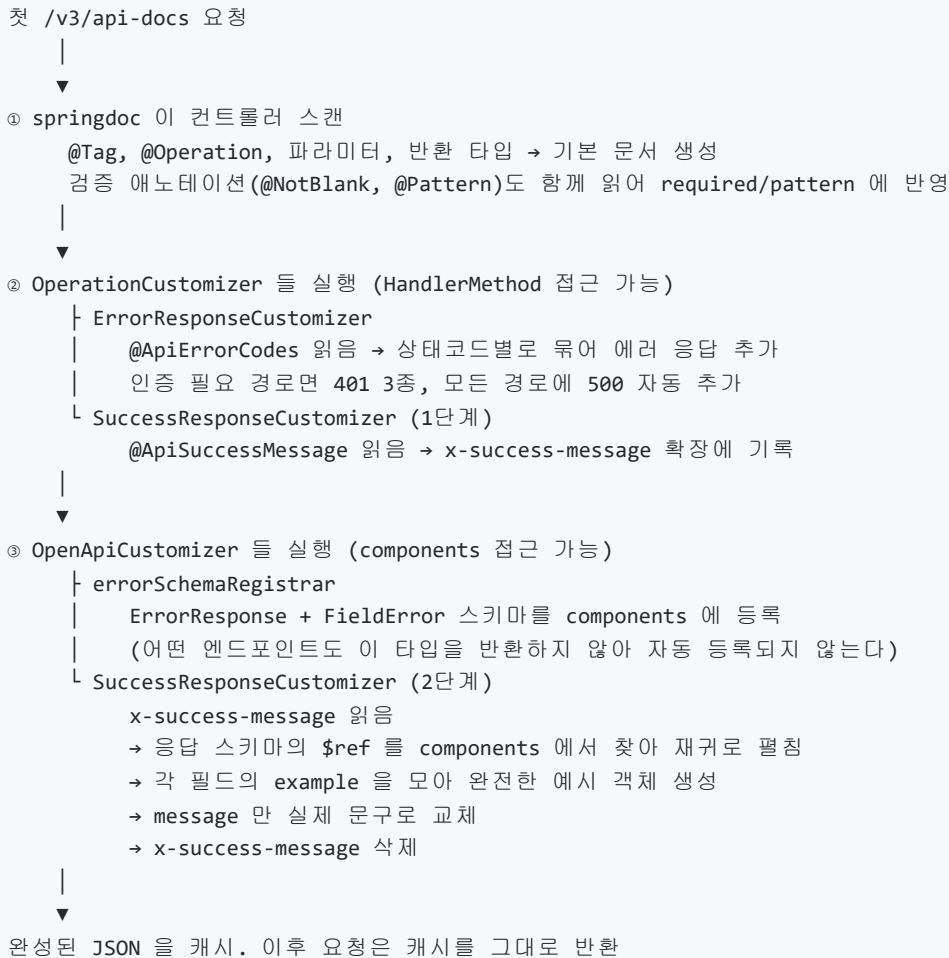
2.9 응답 직렬화 흐름

```
컨트롤러가 ApiResponse<UserResponse> 반환
|
▼
HttpMessageConverter → JsonMapper (Jackson 3)
|
├─ Instant 필드
|   JacksonConfig 가 등록한 InstantSerializer
|   → app.datetime 설정대로 포맷 (기본 yyyy-MM-dd HH:mm:ss, Asia/Seoul)
|
└─ null 필드
    @JsonInclude(NON_NULL) → JSON 에서 제외
    예: message 가 없는 조회 API, data 가 없는 로그아웃
|
▼
{"success":true,"message":"로그아웃되었습니다."}
```

저장은 UTC, 표시만 변환이 원칙이다. 엔티티는 `Instant`, DB 는 `timestampz` 그대로다. 직렬화 순간에만 KST 로 바꾼다.

2.10 Swagger 문서 생성 흐름

문서가 자동으로 만들어지는 과정도 하나의 알고리즘이다.



2단계로 나뉜 이유는 springdoc 의 두 확장 지점이 서로 다른 것을 보기 때문이다.

	볼 수 있는 것	못 보는 것
OperationCustomizer	컨트롤러 애노테이션	components(스키마 정의)
OpenApiCustomizer	문서 전체 + components	애노테이션

message 는 애노테이션에 있고 data 예시는 \$ref 를 따라가야 한다. 한쪽만으로는 안 된다.

3. 코드 리뷰

실제 코드를 읽고 정리한 것이다. 칭찬만 적으면 리뷰가 아니므로 문제도 그대로 적는다. 각 항목에 파일 위치 · 문제 · 재현 조건 · 수정안을 붙였다.

3.1 잘 되어 있는 부분

(1) 계층 간 의존 방향이 지켜진다

`AuthService`, `UserService` 어디에도 HTTP 관련 타입이 없다.

```
// AuthService - HttpServletRequest, ResponseEntity, HttpStatus 가 전혀 없다
throw new BusinessException(Errorcode.INVALID_CREDENTIALS);
```

상태코드 결정은 `Errorcode` 와 `GlobalExceptionHandler` 가 한다. 덕분에 서비스를 배치나 다른 진입점에서 그대로 호출할 수 있고, 단위 테스트에 `MockMvc` 가 필요 없다.

컨트롤러도 얇다. `UserController` 는 전체 58줄이고 메서드 본문이 1~2줄이다. 업무 판단이 하나도 없다.

(2) 에러·성공 문구의 단일 출처

```
Errorcode.INVALID_CREDENTIALS → 401 + "A002" + "이메일 또는 비밀번호가..."
SuccessMessage.SIGNUP         → "회원가입이 완료되었습니다."
```

이 두 enum 만 보면 API 가 내보내는 모든 문구를 알 수 있다. 그리고 문서가 같은 enum 을 참조하므로 응답과 Swagger 가 어긋날 수 없다.

```
@ApiResponse(SuccessMessage.SIGNUP) // 문서
return ApiResponse.ok(..., SuccessMessage.SIGNUP); // 실제 응답
```

문자열을 두 번 적었다면 반드시 언젠가 달라진다. 구조로 막은 것이 좋다.

(3) 보안 기본값이 "차단"

```
.anyRequest().authenticated()
```

화이트리스트에 없는 경로는 자동으로 보호된다. 새 API 를 추가해도 깜빡할 수 없다. 반대 방향(기본 허용 + 보호할 것 나열)이면 빠뜨린 API 가 그대로 뚫린다.

`/me` 경로 설계도 같은 성격이다.

```
@GetMapping("/me") // /api/users/{userId} 가 아님
```

대상이 토큰에서만 결정되므로 IDOR 이 구조적으로 불가능하다. `{userId}` 방식이면 "남의 ID 를 넣었을 때"를 매번 검사해야 하고, 한 번 빠뜨리면 타인 정보가 노출된다.

(4) 엔티티의 변경 지점이 제한적

```
@NoArgsConstructor(access = AccessLevel.PROTECTED) // 빈 객체 생성 차단
@Builder private User(...) // 생성은 빌더로만
public void changeName(String name) // setter 대신 의미 있는 메서드
public void changePassword(String encodedPassword) // 파라미터 이름이 규약을 알려줌
```

`setEmail()` 이 없으므로 이메일은 절대 안 바뀐다. 코드 전체를 검색하지 않아도 보장된다. `changePassword` 를 호출하는 곳을 IDE 로 찾으면 비밀번호가 바뀌는 모든 경로가 나온다.

(5) 스키마 관리가 안전한 쪽으로 설정됨

```
jpa.hibernate.ddl-auto: validate
flyway.enabled: true
```

`update` 는 컬럼 삭제·타입 변경을 하지 않아 반영 안 된 변경이 조용히 쌓인다. `validate` 는 어긋나면 **기동 시점에** 실패한다. 실제로 이 설정이 `@Lob` → Postgres `oid` 매핑 문제를 잡아냈다.

(6) 설정의 비밀값 분리

```
secret: ${JWT_SECRET} # 기본값 없음 → 운영에서 누락 시 기동 실패
url: ${DB_URL:jdbc:postgresql://...} # local 프로파일만 기본값
```

운영에서 환경변수를 깜빡하면 **뜨지 않는다**. 실수로 개발 DB 에 붙어 운영 데이터를 쓰는 사고보다 훨씬 낫다.

3.2 고쳐야 할 것 — 우선순위 순

● A. 회원가입 경쟁 조건이 500 을 낸다

위치 `AuthService.java:39-41` , `GlobalExceptionHandler`

```
if (userRepository.existsByEmail(request.email())) {
    throw new BusinessException(ErrorCode.DUPLICATE_EMAIL);
}
// ... save()
```

문제 확인과 저장 사이에 다른 요청이 같은 이메일로 가입할 수 있다. 그러면 DB 의 `uk_users_email` 제약이 막고 `DataIntegrityViolationException` 이 발생하는데, 이를 처리하는 핸들러가 없어 `Exception` 핸들러로 가서 **500** 이 나간다.

```
요청 A: existsByEmail("a@b.com") → false
요청 B: existsByEmail("a@b.com") → false
요청 A: INSERT → 성공
요청 B: INSERT → 제약 위반 → 500 (409 여야 함)
```

재현 같은 이메일로 동시에 두 번 가입 요청. 확률은 낮지만 0 이 아니다.

수정안 `GlobalExceptionHandler` 에 핸들러 추가.

```

@ExceptionHandler(DataIntegrityViolationException.class)
public ResponseEntity<ErrorResponse> handleDataIntegrity(DataIntegrityViolationException e) {
    log.warn("데이터 무결성 위반", e);
    return ResponseEntity.status(ErrorCode.DUPLICATE_EMAIL.getStatus())
        .body(ErrorResponse.of(ErrorCode.DUPLICATE_EMAIL, ErrorCode.DUPLICATE_EMAIL.getMessage()));
}

```

더 엄밀히 하려면 제약 이름(`uk_users_email`)을 보고 어떤 중복인지 구분해야 한다. 지금은 unique 제약이 이메일 하나뿐이라 단순 처리로 충분하다.

● B. 에러 분기가 메시지 문자열에 의존한다

위치 `JwtAuthenticationEntryPoint.java:35-42`

```

private ErrorCode resolve(AuthenticationException e) {
    if (e instanceof JwtAuthenticationException) {
        String message = e.getMessage();
        return message != null && message.contains("만료") // ← 문자열 비교
            ? ErrorCode.EXPIRED_TOKEN
            : ErrorCode.INVALID_TOKEN;
    }
    return ErrorCode.UNAUTHORIZED;
}

```

문제 `JwtTokenProvider` 의 메시지 문구를 "기간이 지났습니다" 같은 것으로 바꾸면 **조용히 깨진다**. 만료된 토큰에 `A004` 대신 `A003` 이 나가고, 프론트의 자동 재발급 로직이 동작하지 않는다. 컴파일도 테스트도 이걸 못 잡는다.

수정안 예외가 `ErrorCode` 를 직접 들고 다니게 한다.

```

// JwtAuthenticationException
public class JwtAuthenticationException extends AuthenticationException {
    private final ErrorCode errorCode;

    public JwtAuthenticationException(ErrorCode errorCode) {
        super(errorCode.getMessage());
        this.errorCode = errorCode;
    }
    public ErrorCode getErrorCode() { return errorCode; }
}

// JwtTokenProvider
catch (ExpiredJwtException e) {
    throw new JwtAuthenticationException(ErrorCode.EXPIRED_TOKEN);
}

// EntryPoint
private ErrorCode resolve(AuthenticationException e) {
    return e instanceof JwtAuthenticationException jwtE
        ? jwtE.getErrorCode()
        : ErrorCode.UNAUTHORIZED;
}

```

문자열 의존이 사라지고, 새 토큰 에러를 추가할 때 `resolve()` 를 고칠 필요도 없어진다.

● C. 방금 만든 토큰을 다시 파싱한다

위치 `AuthService.java:106` , `AuthService.java:87`

```
refreshTokenRepository.save(RefreshToken.issue(
    user.getId(),
    refreshToken,
    tokenProvider.getExpiresAt(tokenProvider.parse(refreshToken, TokenType.REFRESH))
    //                ↳ 방금 만든 토큰을 서명 검증까지 하며 다시 파싱
));
```

문제 만료 시각을 알려고 자기가 만든 토큰을 도로 검증한다. 동작은 하지만 불필요한 서명 연산이고, 의도가 읽히지 않는다. 같은 패턴이 `reissue()` 에도 있다.

수정안 토큰과 만료 시각을 함께 반환한다.

```
public record IssuedToken(String token, Instant expiresAt) { }
```

```
public IssuedToken createRefreshToken(User user) {
    Instant now = Instant.now();
    Instant expiresAt = now.plus(properties.refreshTokenValidity());
    String token = Jwts.builder()...expiration(Date.from(expiresAt))...compact();
    return new IssuedToken(token, expiresAt);
}
```

호출부가 이렇게 단순해진다.

```
IssuedToken refresh = tokenProvider.createRefreshToken(user);
refreshTokenRepository.save(RefreshToken.issue(user.getId(), refresh.token(), refresh.expiresAt()));
```

● D. 인증된 요청마다 `users` 를 두 번 조회한다

위치 `JwtAuthenticationFilter` + `UserService.getMe()`

```
select ... from users where id = ?    -- ① 필터: 인증 주체 복원
select ... from users where id = ?    -- ② 서비스: 응답 데이터
```

문제 같은 트랜잭션이 아니라서 1차 캐시가 듣지 않는다. `/api/users/me` 는 PK 조회 2회로 처리된다.

판단 지금 규모에서는 **고치지 않아도 된다**. PK 조회는 매우 싸고, 필터의 조회는 탈퇴·권한 변경 즉시 반영이라는 분명한 대가가 있다. 다만 트래픽이 늘면 다음 순서로 손본다.

1. Access 토큰 수명 단축 + 토큰 클레임만 신뢰 (DB 조회 제거)
2. 또는 `AuthUserDetailsService` 에 짧은 TTL 캐시(Caffeine/Redis)

고칠 곳 `AuthUserDetailsService` 한 곳뿐이라는 게 현재 설계의 이점이다.

● E. 활동 로그 적재가 없다

위치 없음 (제거됨)

`ActivityLogger` 는 칸반 도메인과 함께 삭제됐다. 지금은 로그인·로그아웃·비밀번호 변경 같은 보안 이벤트가 아무 데도 기록되지 않는다.

판단 인증 기능만 남은 지금이 오히려 감사 로그가 필요한 시점이다. 프론트에 화면이 없더라도 서버 로그로는 남겨야 한다.

최소 조치 — 테이블 없이 로그만이라도

```
// AuthService.login()
log.info("로그인 성공: userId={}", user.getId());

// UserService.changePassword()
log.info("비밀번호 변경: userId={}", userId);
```

지금은 `AuthService` 에 `@Slf4j` 가 붙어 있는데 로그를 한 줄도 찍지 않는다. 애노테이션만 남은 상태다.

● F. `PageResponse` 가 죽은 코드

위치 `global/response/PageResponse.java`

프로젝트 도메인을 지우면서 참조가 0 이 됐다. 목록 API 가 하나도 없다.

판단 지우거나, 남길 거면 주석으로 이유를 적는다. 죽은 코드는 "이게 왜 있지?" 하는 시간을 계속 잡아먹는다. 칸반 도메인을 곧 다시 만들 계획이면 남겨도 되지만, 그렇다면 주석이 필요하다.

● G. `loadUserByUsername()` 이 호출되지 않는다

위치 `AuthUserDetailsService.java`

```
@Override
public UserDetails loadUserByUsername(String email) { ... } // 호출하는 곳 없음
public AuthUser loadById(UUID userId) { ... } // 실제로 쓰이는 것
```

판단 `UserDetailsService` 인터페이스의 규약이라 구현 자체는 정당하다. Spring Security 의 표준 확장 지점이고, 나중에 `AuthenticationManager` 를 도입하면 쓰인다. 다만 지금 쓰이지 않는다는 사실을 주석으로 남겨두는 편이 읽는 사람에게 친절하다.

● H. `ErrorCode.FORBIDDEN` 의 도달 경로가 좁다

`@PreAuthorize` 를 쓰는 곳이 없어서, `AccessDeniedException` 은 `/actuator/**` 에 일반 사용자가 접근할 때만 발생한다. `GlobalExceptionHandler.handleAccessDenied()` 는 사실상 거의 안 탄다 (`JwtAccessDeniedHandler` 가 필터 단계에서 처리).

판단 문제는 아니다. 관리자 기능이 생기면 바로 쓰인다. 다만 지금 문서에는 403 이 어느 엔드포인트에도 표시되지 않는데, 실제 동작과 일치하므로 맞다.

● 1. 테스트가 사실상 없다

위치 `ApiApplicationTests.java` — `contextLoads()` 하나

```
@Test
void contextLoads() { }
```

"빈 설정이 깨지지 않았다" 만 보증한다. 인증 로직, 권한, 토큰 rotation 은 전혀 검증되지 않는다.

이게 현재 가장 큰 리스크다. 지금까지 확인은 전부 `curl` 수동 테스트였고, 리팩터링하면 다시 손으로 확인해야 한다.

최소 세트 (아래 3.4 에 코드 포함)

1. 로그인 실패 시 `INVALID_CREDENTIALS` 인지 — 사용자 열거 방어를 테스트로 고정
2. Refresh 토큰 rotation 후 이전 토큰이 거부되는지
3. 비밀번호 변경 후 Refresh 토큰이 폐기되는지

3.3 설계 판단 — 지금은 맞지만 언젠가 바뀌야 할 것

리뷰라기보다 "왜 이렇게 했는지"의 기록이다. 면접에서 물어볼 만한 지점들이기도 하다.

현재 선택	이유	바뀌어야 하는 시점
사용자당 Refresh 토큰 1개	단순, 보안 강함	PC·모바일 동시 사용이 불편해질 때 → 디바이스 별 토큰
<code>AuthenticationManager</code> 미사용	흐름이 명확, Security 7 API 변동 회피	계정 잠금(연속 실패 차단)이 필요해질 때
Access 토큰 폐기 불가	무상태 유지	즉시 차단이 필요해질 때 → <code>jti</code> 블랙리스트
날짜에 시간대 표기 없음	한국 사용자만 대상	해외 사용자 → <code>xxx</code> 패턴 또는 ISO
<code>emailVerified = true</code> 고정	SMTP 미구현	이메일 인증 불일 때 (아래 4장)
물리 삭제	단순	탈퇴 복구·감사 요구가 생길 때 → <code>deleted_at</code>
H2 로 테스트	빠름, Docker 불필요	Flyway 마이그레이션까지 검증하려면 → Testcontainers

특히 `emailVerified = true` 는 지금 코드에 `TODO` 로 남아 있다.

```
// TODO 이메일 인증 메일 발송을 붙이면 false 로 두고 인증 후 verifyEmail() 호출
.emailVerified(true)
```

`User.verifyEmail()` 메서드는 이미 만들어져 있고 **호출되는 곳이 없다**. SMTP 만 붙이면 되는 상태다.

3.4 테스트 작성 예시

가장 시급한 항목이라 바로 쓸 수 있게 적어둔다.

단위 테스트 — 사용자 열거 방어

```
@ExtendWith(MockitoExtension.class)
class AuthServiceTest {

    @Mock UserRepository userRepository;
    @Mock RefreshTokenRepository refreshTokenRepository;
    @Mock PasswordEncoder passwordEncoder;
    @Mock JwtTokenProvider tokenProvider;

    @InjectMocks AuthService authService;

    @Test
    void 없는_이메일로_로그인해도_USER_NOT_FOUND가_아니라_INVALID_CREDENTIALS를_던진다() {
        given(userRepository.findByEmail(any())).willReturn(Optional.empty());

        assertThatThrownBy(() -> authService.login(new LoginRequest("none@example.com", "pw")))
            .isInstanceOf(BusinessException.class)
            .extracting("errorCode")
            .isEqualTo(ErrorCode.INVALID_CREDENTIALS);
    }

    @Test
    void 중복_이메일로_가입하면_DUPLICATE_EMAIL() {
        given(userRepository.existsByEmail("dup@example.com")).willReturn(true);

        assertThatThrownBy(() ->
            authService.signup(new SignupRequest("dup@example.com", "Yudillo!234", "홍길동")))
            .isInstanceOf(BusinessException.class)
            .extracting("errorCode")
            .isEqualTo(ErrorCode.DUPLICATE_EMAIL);
    }
}
```

첫 번째 테스트가 특히 의미 있다. "사용자 열거 방어"라는 의도를 테스트로 고정한다. 나중에 누군가 친절하게 `USER_NOT_FOUND` 로 바꾸려 하면 테스트가 막아준다. 주석보다 강한 장치다.

통합 테스트 — 토큰 rotation

```
@ActiveProfiles("test")
@SpringBootTest
@AutoConfigureMockMvc
class AuthFlowIntegrationTest {

    @Autowired MockMvc mockMvc;
    @Autowired ObjectMapper objectMapper;

    @Test
    void 재발급_후_이전_refreshToken_은_거부된다() throws Exception {
        // given: 가입 + 로그인
        signup("rot@example.com", "Yudillo!234", "회전");
        String oldRefresh = login("rot@example.com", "Yudillo!234").refreshToken();

        // when: 재발급
        mockMvc.perform(post("/api/auth/reissue")
            .contentType(APPLICATION_JSON)
            .content("""
                {"refreshToken": "%s"}""".formatted(oldRefresh)))
            .andExpect(status().isOk());

        // then: 같은 토큰으로 다시 시도하면 거부
        mockMvc.perform(post("/api/auth/reissue")
            .contentType(APPLICATION_JSON)
            .content("""
                {"refreshToken": "%s"}""".formatted(oldRefresh)))
            .andExpect(status().isUnauthorized())
            .andExpect(jsonPath("$.code").value("A003"));
    }
}
```

18장에서 `curl` 로 확인했던 시나리오를 그대로 옮기면 된다. 이미 통과하는 걸 알고 있으니 테스트를 쓰는 부담이 적다.

3.5 한눈에 보는 리뷰 결과

#	항목	심각도	위치	예상 작업
A	경쟁 조건 시 500 응답	●	<code>GlobalExceptionHandler</code>	10분
B	메시지 문자열로 에러 분기	●	<code>JwtAuthenticationEntryPoint</code>	20분
C	토큰 재파싱	●	<code>AuthService</code> , <code>JwtTokenProvider</code>	30분
D	요청당 <code>users</code> 2회 조회	●	필터 + 서비스	지금은 유지
E	보안 이벤트 로그 없음	●	<code>AuthService</code> , <code>UserService</code>	10분
F	<code>PageResponse</code> 죽은 코드	●	<code>global/response</code>	2분
G	<code>loadUserByUsername</code> 미사용	●	<code>AuthUserDetailsService</code>	주석만
H	<code>FORBIDDEN</code> 도달 경로 좁음	●	-	조치 불필요
I	테스트 부재	●	<code>src/test</code>	2~3시간

● 4건 중 A-B 는 각각 30분 이내이고, I(테스트)가 실질적으로 가장 큰 일이다.

4. 권장 작업 순서

4.1 코드 품질 (반나절)

- | | |
|--|-----|
| 1. <code>GlobalExceptionHandler</code> 에 <code>DataIntegrityViolationException</code> 핸들러 추가 | [A] |
| 2. <code>JwtAuthenticationException</code> 이 <code>ErrorCode</code> 를 늘도록 변경 | [B] |
| 3. <code>AuthService</code> / <code>UserService</code> 에 보안 이벤트 로그 추가 | [E] |
| 4. <code>PageResponse</code> 처리 (삭제 또는 주석) | [F] |
| 5. 위 변경을 테스트로 고정 | [I] |

1~4 를 먼저 하고 5 를 하는 순서를 권한다. 고칠 것을 다 고친 뒤 테스트를 쓰면 테스트를 두 번 고치지 않는다.

4.2 기능 (프론트와 맞물림)

6. 이메일 인증 / 비밀번호 재설정
V3__add_email_verification_tokens.sql
MailSender 인터페이스 + @Profile("local") 콘솔 구현
/api/auth/email/**, /api/auth/password/** (SecurityConfig 에 자리 예약됨)
7. 프론트엔드 마이그레이션
src/api/client.ts - 토큰 첨부 + A004 자동 재발급
src/supabase/ 제거
8. 칸반 도메인 재구축 (프론트 화면 생긴 뒤)
boardlist → card → activity 순서
position 은 double (fractional indexing)

6번의 MailSender 패턴은 적어둘 만하다.

```
public interface MailSender { void send(String to, String subject, String body); }

@Component @Profile("local")
public class LoggingMailSender implements MailSender {
    public void send(...) { log.info("=== MAIL ===\nTo: {}\n{}", to, body); }
}

@Component @Profile("prod")
public class SmtplibMailSender implements MailSender { ... }
```

로컬에서는 콘솔에 인증 링크가 찍히니 SMTP 없이 흐름을 완성할 수 있다. 결제·SMS·파일 저장 등 외부 연동에 두루 쓰는 방식이다.

4.3 배포 (포트폴리오 마무리)

9. Dockerfile + docker-compose.prod.yml
10. README 에 배포 구조 다이어그램
11. 실제 배포 (Railway 또는 Oracle Cloud Free)

면접에서 "어떻게 배포하셨어요?"는 거의 항상 나온다. 동작하는 URL 이 있으면 설명이 훨씬 쉬워진다.

5. 참고 — 이 프로젝트에서 설명할 수 있는 것

포트폴리오 관점에서, 지금 코드로 답할 수 있는 질문들이다.

질문	답할 재료
인증을 어떻게 구현했나요?	2.3 ~ 2.6. Access/Refresh 분리, rotation, DB 저장 이유
JWT 의 단점은?	폐기 불가. 그래서 Access 는 짧게, Refresh 만 DB 에
왜 Supabase 를 걷어냈나요?	클라이언트 직접 DB 접근 → 접근 제어가 DB 정책에 묶임
로그인 실패 메시지를 왜 통일했나요?	사용자 열거 방어 (2.3)
스키마 관리는?	Flyway + <code>validate</code> . <code>update</code> 가 위험한 이유 (3.1-5)
예외 처리 구조는?	2.8. 필터 단계와 컨트롤러 단계의 처리 주체가 다름
개선할 점은?	3.2 전체. 자기 코드의 약점을 아는 것이 오히려 점수가 된다

마지막 항목이 중요하다. "완벽합니다"보다 "여기 경쟁 조건이 있어서 409 대신 500 이 나갑니다, 핸들러 추가로 고칠 수 있습니다"가 훨씬 신뢰를 준다.

문서 끝. 2026-09-28 기준 코드.